Analyzing and Predicting Ram Price Trends Using Machine Learning

Anson Lau, Anthony Pham Kieu

SJSU CS 133

4/24/2026

Table Of Contents

1. Abstract **3**

2. Background & Problem Statement **3**

3. Program Design & Organization **4**

4. Exploratory Data Analysis (EDA) **5**

5. Machine Learning **7**

6. Model Evaluation, Test Strategy, & Results **8**

7. Conclusion **9**

8. Instruction to Run Code **10**

9. References **11**

1. Abstract

This project analyzes RAM prices trends and sees whether factors such as RAM generation, brand, and capacity affects price. We use the dataset of historical RAM listings to perform exploratory data analysis (EDA) and build machine learning models to predict RAM prices. EDA graphs include scatter plots, violin plots, and boxplots to analyze pricing trends. The models we use are linear regression with polynomial features, decision tree regressor, and random forest regressor. We tested these using a method called N-fold cross-validation, along with a few other testing metrics. Our models were able to perform decently well with the given dataset. Overall, these visualizations show that newer RAM generations have higher price variability and generally higher cost.

2. Background & Problem Statement

Recently, social media has shown that RAM prices are increasing due to rising demand for AI. This project investigates whether these claims are true by using datasets for various RAM factors. Our goal is to identify key factors of RAM pricing and build a model for market trends. This will help consumers time the market potentially to get the best value for their money.

The dataset we used in this project comes from a public GitHub repository containing historical RAM listings. The data includes tables with information like price per gigabyte, memory size, brand, ram generation, form factor, and date of listing. We chose this dataset because it reflects the real-world market and is good for predicting what will happen in the future.

The following are research questions that helped guide our data exploration.

Research Questions:

1. Have there been significant price spikes/drops in RAM?
2. What factors contribute most to price increases?
3. How do RAM generations compare in price? (DDR3–DDR5)

4. How does form factor affect price? (laptop, server, or desktop)
5. How do different brands compare?

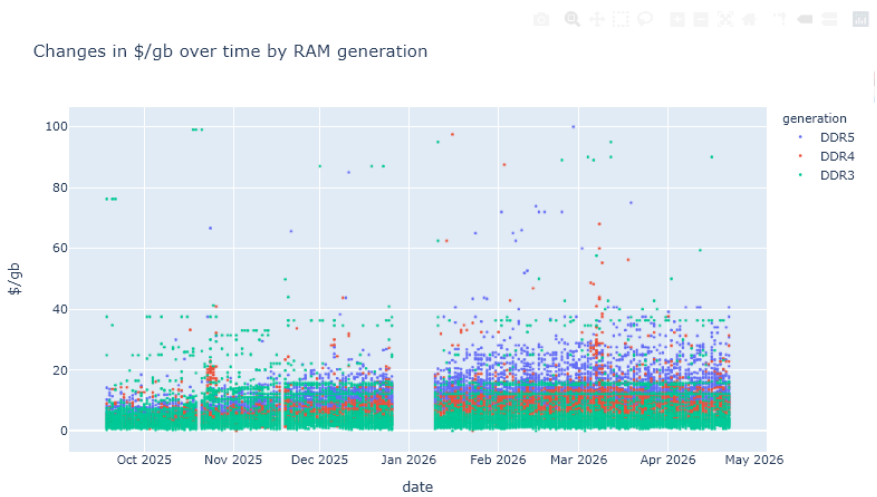
3. Program Design & Organization

When designing, we wanted to ensure clarity and reproducibility. To do this, we implemented scikit-learn (sklearn) which allows for different stages of the machine learning process to be organized effectively. There is also a separation between training and test data to prevent data leakage and ensure unbiased model testing.

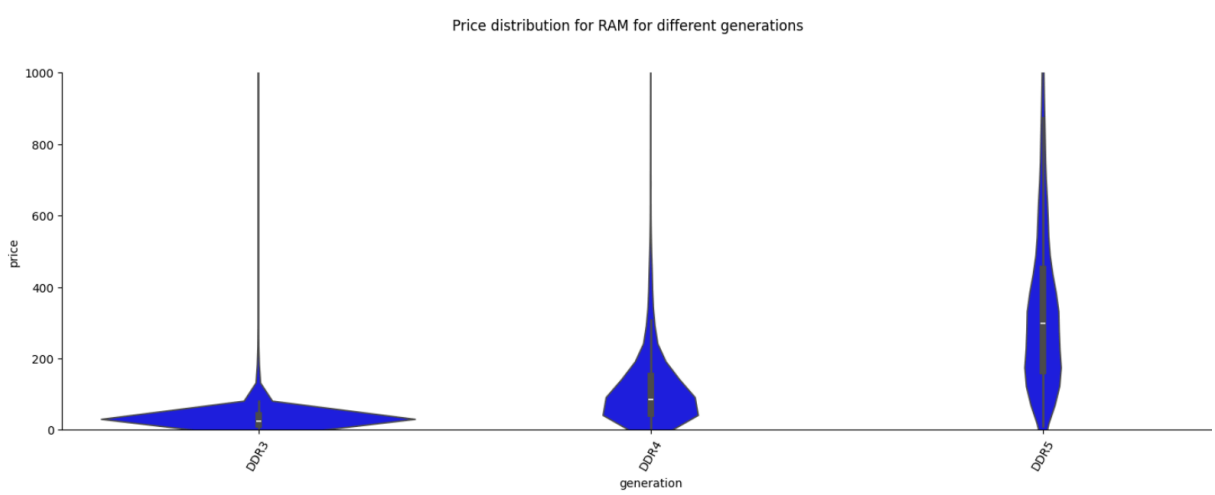
The program is organized into several stages. First, the datasets are loaded by combining multiple JSON files from the original GitHub repository into a single dataset. Each raw file contained information for a specific day, and so we merged them to have the data be easily accessible in a single file. Since this dataset was very large (near 300,000 elements), we pulled a random sample of 30,000 rows. This helped improve processing time while also still having representative results of the larger dataset. Next, we cleaned the data by removing unnecessary columns such as source, currency, and links to the sale. The price column was converted into a numeric format to make sure there is compatibility with the machine learning models. We also removed outliers where prices were greater than 6,000 since such RAM isn't going to be important for the average user. We also decided to remove rows with null values, since there were only 3 rows which wouldn't really affect the dataset. Next, we did some simple data analysis and visualization to find patterns in RAM pricing. This included creating categories for RAM generations, analyzing price trends over time, and comparing different features such as brand, capacity, and form factor. Finally, the processed data is used for model training and evaluation.

4. Exploratory Data Analysis (EDA)

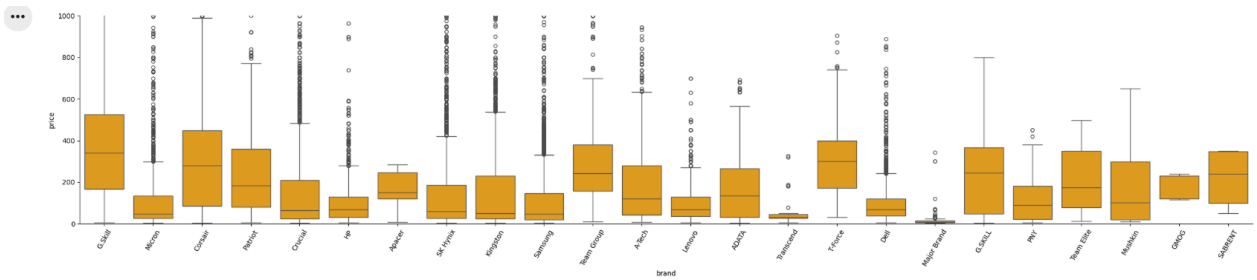
One of the primary graphs we used was a scatter plot which shows the relationship between price per gigabyte ($/GB) and time, separated by RAM generation (DDR3, DDR4, and DDR5). This plot showed that DDR5 and DDR4 RAM are consistently more expensive than DDR3 and also helped identify a spike in DDR4 prices around March 2026, which helped us answer question 1 and 3.



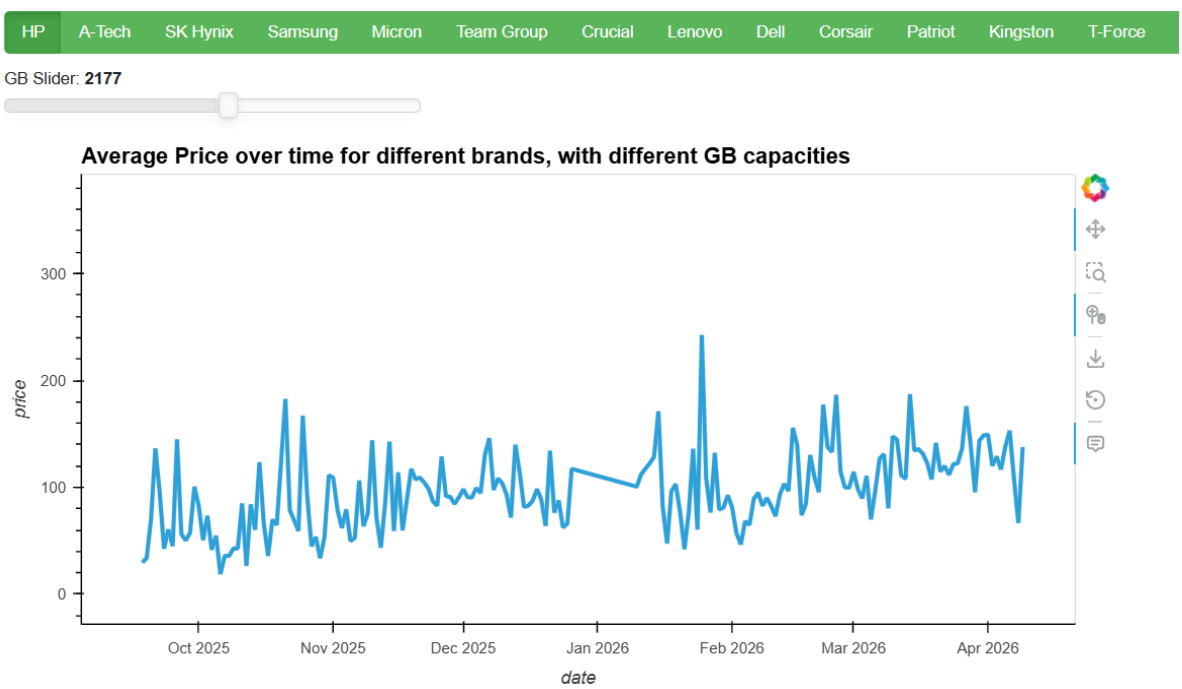
On a related note, another particularly useful graph that we created revolving around RAM generations was a violin plot. We were able to see how the "$/gb" distribution differed for different RAM generations. DDR3 had a large concentration of values with lower $/gb. DDR4 had a solid concentration in low to mid-tier prices. DDR5 had a wide, even distribution from low prices to extremely high prices. This graph corroborates the findings from the previous graph, showing that RAM generation has a strong effect on RAM prices (Question 3).



Another particularly useful graph for us was the boxplot displaying the relationship between the price and the brands. We found that certain brands had a higher average price than other brands. The most expensive brand on average was G. Skill (Question 5).

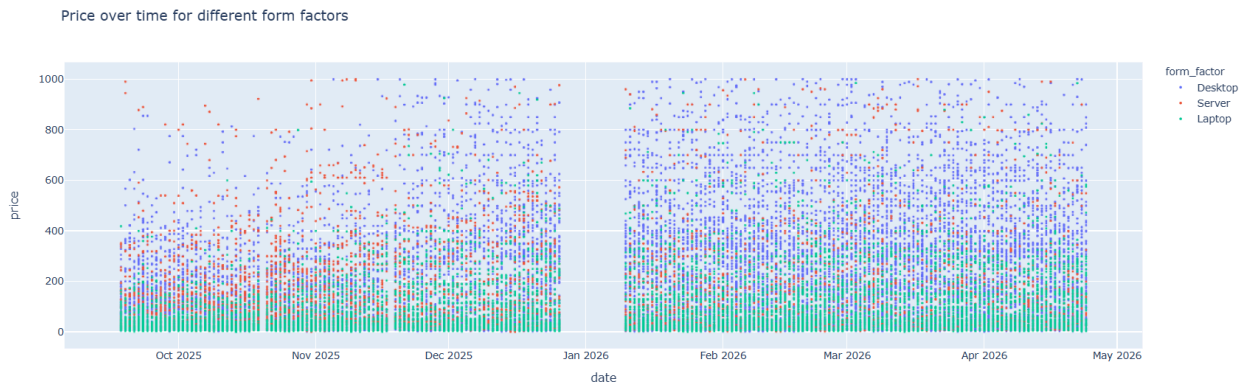


Another important graph we made was an interactive dashboard built using Panel. This dashboard allowed users to filter data by brand and RAM capacity, allowing for more flexibility when exploring price trends. Through this graph, we were able to see prices over time for different brands and see short-term spikes in the market (Question 5).



A graph that helped us answer question 4 was an interactive scatter plot that gave us information about the price distribution for different form factors of RAM (server, laptop, desktop). The graph (especially when toggling individual form factors), shows that RAM for servers and laptops tend to be

heavily concentrated at lower prices, while desktop RAM on average has a much higher range of prices. This indicates that a strong predictor of RAM prices is the form factor, as RAM for desktop tend to have higher prices.



5. Machine Learning

To prepare the dataset for training and testing, we used a method from Scikit-learn called StratifiedShuffleSplit. This method splits the data into two parts: 80% for training and 20% for testing. To ensure that both the training and testing have similar distribution of RAM prices, we stratified based on the given price column. The features we used for model training are divided into numerical and categorical variables. The numerical features are memory capacity, date, and price per gigabyte ($/GB). The categorical features are brand, RAM generation, form factor, and condition.

```
num_attribs = ["$/gb", "capacity_gb", "date"]
cat_attribs = ["brand", "generation", "form_factor", "condition"]
```

After the data split, we used Scikit-learn to create a machine learning pipeline. We used StandardScaler to adjust the numerical data to a uniform scale and OneHotEncoder to convert the categorical features into a format the models can read. Then, we combined these using a ColumnTransformer to ensure that both numerical and categorical data were processed before being passed into the models.

Next, we used three machine learning models to predict RAM prices. The first model was a polynomial linear regression model. This model helped us find potential non-linear relationships between features and price which is better than a standard linear regression model.

```
polynomial = PolynomialFeatures(degree=2, include_bias=False)
poly_ram_prepared = polynomial.fit_transform(ml_ram_prepared)
lin_reg = LinearRegression()
lin_reg.fit(poly_ram_prepared, ram_training_labels)
```

The second model was a decision tree regressor. This model helped us find out how features interact with each other. Decision trees are good at modeling relationships that are not linear and can automatically find the most important features.

```
tree_reg = DecisionTreeRegressor(random_state=50)
tree_reg.fit(ml_ram_prepared, ram_training_labels)
```

The final model is a random forest regressor. We used this model to improve our predictions and reduce overfitting. By combining multiple decision trees, the random forest model has better predictions compared to a single decision tree.

```
params = [
    {'n_estimators': [50, 100], 'max_features': [3, 7]},
    {'bootstrap': [False], 'n_estimators': [50, 100], 'max_features':
[3,7]},
  ]

for_reg = RandomForestRegressor(random_state = 50)
grid_search = GridSearchCV(for_reg, params, cv=5,
                           scoring='neg_mean_squared_error',
return_train_score=True)
grid_search.fit(ml_ram_prepared, ram_training_labels)
```

6. Model Evaluation, Test Strategy, & Results

Since we are trying to predict prices, we chose to use regression models. Metrics like accuracy and precision are less useful due to the fact that the models are regression models, so we used three other metrics. These metrics are RMSE (Root Mean Squared Error), MAE (Mean Absolute Error), and 10-fold

cross validation using the RMSE metric. RMSE and MAE were calculated using the test set, and the cross validation scores were calculated using the training set.

For the polynomial linear regression model, we obtained errors of:

```
cross validation mean (rmse):  48.4726919339665
lin_rmse: 4.531059100174376
lin_mae: 2.5097771739567074
```

The low RMSE and MAE scores compared to the cross validation mean indicate that the model may have slightly overfitted. Taking a look at the most reliable metric, the cross validation score shows that the predictions are off by 48.47 dollars. Compared to the standard deviation of 273.93, this error isn't too bad. However, it can be improved.

For the decision tree model, we obtained errors of:

```
cross validation mean (rsme):  34.404162471180925
dec_rmse: 59.00542012634861
dec_mae: 2.90204336947459
```

The high RMSE compared to the low MAE shows that the model is poorly predicting on outliers, but that in general it can predict pretty well. Again, looking at the mean of the cross validation scores, this model performs much better than the linear regression model. On average, it is off by 34.40 dollars, which is an improvement and only 12% of the standard deviation.

For the random forest regression model, we got a final cross validation mean of 72.67. This model was very time consuming since we decided to do grid search with it to find the best hyperparameters, but we weren't able to obtain a model that was better than the other two.

7. Conclusion

Through our research, we were able to get a better understanding of RAM prices. We were able to find strong insights from the dataset that we chose. In particular, we found:

- There was a large spike in prices in March 2026 for DDR4 RAM (Question 1)
- RAM generation significantly affects the RAM price, with DDR5 having a higher price spread and DDR3 having a large concentration of values with lower prices (Question 3)
- RAM form factor also significantly affects RAM price. Server and laptop RAM is concentrated at lower prices while desktop RAM has prices consistently ranging anywhere from low to high.
- Different brands have different prices on average, with some brands costing more than others. The most expensive brand was G. Skill (Question 5)
- RAM generation, form factor, and brand are all important factors to consider when trying to predict RAM prices (Question 2)

We were also able to create a couple of different models and get the models to predict RAM prices with the given information. Although our models weren't perfect, they were pretty decent at predicting RAM prices. Our best model, the Decision Tree Regression Model, was able to predict prices with a cross validation mean error of 34.40, only 12% of the standard deviation.

8. Instruction to Run Code

1. Install dependencies:

```
pip install pandas numpy scikit-learn plotly panel hvplot
```

2. Clone dataset:

```
git clone https://github.com/HardwareDealsCo/ram-deals.git
```

3. Run notebook:

```
jupyter notebook Project_Machine_Learning.ipynb
```

4. Execute all cells in order

9. References

- RAM Dataset: https://github.com/HardwareDealsCo/ram-deals
- Historical Memory Data: https://ourworldindata.org
- Scikit-learn Documentation: https://scikit-learn.org